



JavaScript

Урок №17: Основы.Деструктуризация.

Автор Бахшиллоев Бехзод

Ментор по Frontend в **PRO** **UNITY**



Деструктурирующее присваивание. Массивы.

Деструктурирующее присваивание - способ извлечения данных из массива или объекта специальным синтаксисом.

Для извлечения данных из массива используются квадратные скобки []:

```
const [ name1, name2 ] = [ 'John', 'Mike', 'Abraham', 'Piter' ];  
console.log(name1, name2); // John Mike
```

Для того, чтобы получить элементы из середины массива, нужно в деструктурирующем присваивании пропустить ненужные элементы:

```
const [ , name2, name3 ] = [ 'John', 'Mike', 'Abraham', 'Piter' ];  
console.log(name2, name3); // Mike Abraham
```

Здесь мы оставили пустой первую переменную, указав лишь запятую.

[Ссылка 1](#)

[Ссылка 2](#)



Деструктурирующее присваивание. Массивы.

Если воспользуемся **rest** оператором, можем получить остаток массива в переменную:

```
const [ name1, name2, ...other ] = [ 'John', 'Mike', 'Abraham', 'Piter' ];  
console.log(name1, name2); // John Mike  
console.log(other); //["Abraham", "Piter"]
```

rest оператор в деструктурирующем приравнении может быть только в конце выражения (перед закрывающей квадратной скобкой).

Если в деструктурирующем присваивании больше переменных, чем элементов в массиве, то последним переменным, которым “не хватило” элементов из массива, будет присвоено `undefined`:

```
const [ name1, name2, name3 ] = [ 'John', 'Mike' ];  
console.log(name1, name2, name3); // John Mike undefined  
Однако этого можно избежать, используя значения по умолчанию:
```

```
const [ name1, name2, name3 = "Unknown" ] = [ 'John', 'Mike' ];  
console.log(name1, name2, name3); // John Mike Unknown
```



Деструктурирующее присваивание. Объекты.

Работа с деструктурирующим присваиванием на объектах производится аналогично работе с массивами.

Для извлечения данных из объекта путем деструктуризации используются фигурные скобки `{}` и имена переменных, соответствующие полям объекта:

```
const figure = { width: 10, height: 12, type: 'square' };  
const { width, height, type } = figure;
```

```
console.log(width, height, type); // 10 12 "square"
```

При необходимости использовать другие имена переменных в деструктуризации используется двоеточие:

```
const figure = { width: 10, height: 12, type: 'square' };  
const { width: w, height: h, type: t } = figure;  
console.log(w, h, t); // 10 12 "square"
```



Деструктурирующее присваивание. Объекты.

Destructuring также позволяет использовать свойства по умолчанию:

```
const info = { type: 'html' };  
const { type, size = 0 } = info;  
console.log(type, size) // 'html' 0
```

При использовании деструктурирующего присваивания в правой части выражения можно использовать любые валидные операции, предоставляющие объект:

```
function getUser() {  
  return { name: 'John', age: 15 };  
}  
const { name, age } = getUser();  
console.log(name, age); // John 15
```

Деструктурирующее присваивание. Объекты.

Destructuring позволяет присваивать и вложенные объекты, для этого используется вложенные фигурные скобки:

```
const element = {  
  name: 'div',  
  attributes: { className: 'box', title: 'info' }  
};
```

```
const { name, attributes: { className, title } } = element;  
console.log(name, className, title);  
// "div" "box" "info"
```

Destructuring позволяет комбинировать присваивание объектов, внутри которых вложены массивы:

```
const element = {  
  name: 'div',  
  children: [ { name: 'span' }, { name: 'em' } ]  
};  
const { name: parentEl, children: [ child1 ] } = element;  
console.log(parentEl, child1); →  
// "div" Object {name: "span"}
```



Деструктурирующее присваивание.

Можно деструктурировать сущности любой сложности:

```
const element = {  
  name: 'div',  
  children: [ { name: 'span' }, { name: 'em' } ]  
};  
const { children: [ { name } ] } = element;  
console.log(name); // "span"
```



Деструктурирующее присваивание. Функции.

Destructuring можно использовать внутри функции при работе с аргументами:

```
const user = { name: 'Louis', surname: 'Watkins', gender: 'Male', age: 50 };
```

```
function getBaseInfo( { name, age } ) {  
  return `name - ${name}; age - ${age}`;  
}  
getBaseInfo(user); // "name - Louis; age - 50"
```

Здесь также возможны значения по умолчанию:

```
function getBaseInfo( { name = 'John', age = 18 } ) {  
  return `name - ${name}; age - ${age}`;  
}  
getBaseInfo({}); // "name - John; age - 18"
```

Однако вызов без аргумента приведёт к ошибке:

```
getBaseInfo(); →
```

```
Uncaught TypeError: Cannot destructure property `name` of 'undefined' or 'null'
```




Деструктурирующее присваивание. Функции.

Для того, чтобы ошибка не возникала, нужно указать функции, что если ничего не передано, то использовать по умолчанию пустой объект:

```
function getBaseInfo( { name = 'John', age = 18 } = {} ) {  
  return `name - ${name}; age - ${age}`;  
}  
getBaseInfo(); // "name - John; age - 18"
```



Задачи часть 1:

1. Используя rest оператор и деструктуризацию, создать функцию, которая принимает любое количество аргументов и возвращает объект, содержащий первый аргумент и массив из остатка:

```
func('a', 'b', 'c', 'd') →  
{  
  first: 'a',  
  other: ['b', 'c', 'd']  
}
```



Задачи часть 2:

2. Организовать функцию `getInfo`, которая принимает объект вида `{ name: ..., info: { employees: [...], partners: [ ... ] } }` и выводит в консоль имя (если имени нет, показывать 'Unknown') и первые две компании из массива `partners`:

```
const organisation = {  
  name: 'Google',  
  info: { employees: ['Vlad', 'Olga'], partners: ['Microsoft', 'Facebook',  
    'Xing'] }  
};  
getInfo(organisation); →  
Name: Google  
Partners: Microsoft Facebook
```